

Abstract Data Types

An *abstract data type* (ADT) is a specification of a set of data and the set of operations that can be performed on the data. Such a data type is abstract in the sense that it is independent of various concrete implementations. The definition can be *mathematical*, or it can be programmed as an *interface*.

When realized in a Java program, the ADT is represented by an **interface**, which shields a corresponding implementation. Users of an ADT are concerned with the interface, but not the implementation, as the implementation can change in the future. (This supports the principle of information hiding, or protecting the program from design decisions that are subject to change.)

The strength of an ADT is that the implementation is hidden from the user. Only the interface is published. This means that the ADT can be implemented in various ways, but as long as it adheres to the interface, user programs are unaffected. For example, a **List** ADT can be represented using an array-based implementation or a linked-list implementation.

Java provides interfaces and implementations for many abstract data types. They are in the package `java.util`, and are known as the *Collections Framework*. Fig. 1 shows the main Collection interfaces in `java.util`.

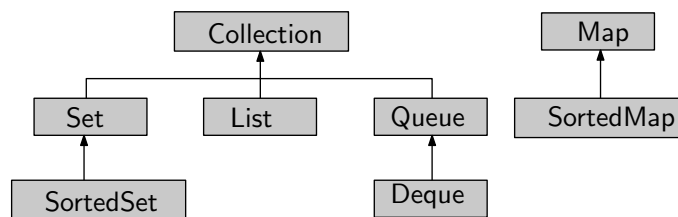


Figure 1: Java collection interfaces

Map

A *map* (also called a *dictionary*) is an object that maps *keys* to *values*. A map supports two main operations:

- insert a (key, value) pair,
- given a key, find the value

A map cannot contain duplicate keys.

```
public interface Map<Key, Value> {
    boolean isEmpty();
    boolean containsKey(Key key);
    Value get(Key key);
    Value put(Key key, Value value);
    Value remove(Key key);
}
```

There is also a **SortedMap** interface, which can be used for elements that can be compared:

```
public interface SortedMap<Key, Value> extends Map<Key, Value> {

    Key firstKey();
    Key lastKey();
    SortedMap<Key, Value> subMap(Key fromKey, Key toKey);
}
```

The Java class `TreeMap` implements `SortedMap` using a balanced binary tree, and operations typically take $O(\log n)$ time. The class `HashMap` implements `Map` using a hash table, and operations take constant time.

`HashMap` requires that the key objects implement the `hashCode` method correctly: if two key objects are equal, then they must have identical `hashCode`. This is true for all standard types, but needs to be done by the programmer for new types!

Collection

A *collection* represents a group of objects, known as its *elements*.

```
public interface Collection<AnyType> {
    void clear();
    boolean isEmpty();
    int size();
    boolean add(AnyType obj);
    boolean contains(Object obj);
    boolean remove(Object obj);
    Iterator<AnyType> iterator();
}
```

Java does not provide any direct implementations of the `Collection` interface.

Set

A *set* is a collection that contains no duplicate elements. As implied by its name, this interface models the mathematical *set* abstraction. A `SortedSet` is a set where some order of its elements exists.

```
public interface Set<T> extends Collection<T> {
    // no new methods
}
public interface SortedSet<T> extends Set<T> {
    T first();
    T last();
    SortedSet<T> subSet(T fromElem, T toElem);
}
```

Implementations of `Set` and `SortedSet` are similar to implementations of `Map` and `SortedMap` — in fact, a `Set` can simply be implemented using a `Map`. `java.util` contains the `TreeSet` class implementing the `SortedSet` interface using a balanced binary tree, and the `HashSet` class implementing the `Set` interface using a hash table. As for `HashMap`, objects stored in a `HashSet` must correctly implement `hashCode`!

List

The name “list” is a bit confusing, because in computer science “list” is very often used to mean “linked-list”. In Java, however, a `List` is an abstract data type with the meaning of a *sequence*. A `List` stores a sequence of elements, and the user of the `List` has precise control over where in the sequence each element is inserted. A `List` (sequence) can be implemented using a linked-list, but other implementations are also possible (for instance using an array).

`List` elements can be accessed by their integer index. Unlike sets, lists typically allow duplicate elements.

```
public interface List<T> extends Collection<T> {
    T get(int idx);
    T set(int idx, T obj);
}
```

```

    int indexOf(Object obj);
    int lastIndexOf(Object obj);
    T remove(int idx);
    void add(int idx, T obj);
    List<T> subList(int fromIdx, int toIdx);
}

```

The two main implementations of `List` in the Java collections framework are `ArrayList` (which implements a sequence using a dynamic array) and `LinkedList` (which uses a linked list, of course). `ArrayList` provides fast access by index, and allows for fast appending. It is slow for inserting and deleting elements in the middle. `LinkedList` is fast for adding and removing elements, but slow for accessing elements by index.

Container iterations

One common operation that we often want to perform on a collection is *iteration*, namely to look at all the elements of the collection.

Iterators encapsulate the concept of iteration. This allows us to write the same iteration code for any kind of collection, and to freely change the collection that provides the elements to be considered.

The elements of an array can be listed like this:

```

for (int i = 0; i < v.length; ++i)
    System.out.println(v[i]);

```

On linked lists, it is not a good idea to access elements by their indices, because it takes $O(n)$ time to access an element by its index. By providing an iterator object, a collection can provide an efficient way of listing the elements without giving access to implementation details.

Java iterators work like this:

```

for (it = v.iterator(); it.hasNext(); )
    System.out.println(it.next());

```

Alternatively, we can write the following short Java code. The Java compiler converts it into the code above.

```

for (ElementType e : v)
    System.out.println(e);

```

To access the contents of a `Map<Key, Value> map`, obtain an iterator for its set of keys using

```

Iterator<Key> it = map.keySet().iterator()

```

Adding iterators to your own containers

Every Java collection provides an iterator object that implements the `Iterator` interface. We can implement the `Iterator` interface on our *own* collection classes as well.

Consider the `ArrayList` class we made earlier. First, `ArrayList` must now implement the `Iterable` interface, so that the Java compiler knows that `ArrayList` objects provide iterators. It then remains to implement the iterator class (see `ALIterator`).